

Введение

Огромная часть науки о данных включает в себя сбор необработанных данных и преобразование их в форму, готовую для анализа. По некоторым оценкам, специалисты по обработке данных тратят 80% своего времени на очистку данных и манипулирование ими, и только 20% своего времени фактически анализируют или строят на их основе модели.

Когда мы получаем необработанные данные, мы должны сделать ряд вещей, прежде чем мы будем готовы их проанализировать, включая:

- диагностика «чистоты» данных - сколько очистки данных нам придется сделать
- изменение формы данных - получение правильных строк и столбцов для эффективного анализа
- объединение нескольких файлов
- изменение типов значений - как мы фиксируем столбец, в котором числовые значения хранятся в виде строк, например
- удаление или заполнение отсутствующих значений - как мы поступаем с неполными или отсутствующими данными
- манипулирование строками для лучшего представления данных

Мы рассмотрим методы, которые используют специалисты по данным для достижения этих целей, просмотрев некоторые «нечистые» наборы данных и попытавшись привести их в хорошее, чистое состояние.

Диагностировать данные

Мы часто описываем данные, которые легко анализировать и визуализировать, как «аккуратные данные». Что значит иметь аккуратные данные?

Чтобы данные были аккуратными, они должны соответствовать следующим условиям:

- Каждая переменная должна быть отдельный столбец
- Каждая строка как отдельное наблюдение

Например, мы хотели бы изменить форму таблицы следующим образом:

Account	Checkings	Savings
“12456543”	8500	8900
“12283942”	6410	8020
“12839485”	78000	92000

В таблицу, которая больше похожа на:

Account	Account Type	Amount
“12456543”	“Checking”	8500
“12456543”	“Savings”	8900
“12283942”	“Checking”	6410
“12283942”	“Savings”	8020

Account	Account Type	Amount
"12839485"	"Checking"	78000
"12839485"	"Savings"	920000

Первый шаг диагностики аккуратности набора данных - использование функций pandas для исследования и проверки набора данных.

Вы видели большинство функций, которые мы часто используем для диагностики набора данных для очистки. Вот некоторые из самых полезных:

- `.head ()` - отобразить первые 5 строк таблицы
- `.info ()` - отобразить сводку таблицы
- `.describe ()` - отображать сводную статистику таблицы
- `.columns` - отображать имена столбцов таблицы
- `.value_counts ()` - отображать отдельные значения для столбца

Задание

1. Мы предоставили два DataFrames, `df1` и `df2`.

Проверьте оба этих DataFrames, используя перечисленные выше функции.

Начните с печати `.head ()` как `df1`, так и `df2`.

В пандас можно работать с файлами excel

```
pd.ExcelFile('ШП.xlsx').parse('Sheet1')
```

2. Изучите DataFrames с помощью других перечисленных функций.

Какой DataFrame является «чистым» и готовым к анализу? Создайте переменную с именем `clean` и установите для нее значение 1 для `df1` и 2 для `df2`.

Работа с несколькими файлами

Часто одни и те же данные разделены на несколько файлов.

Допустим, у нас есть масса файлов, следующих по структуре имен файлов: `file1.csv`, `file2.csv`, `file3.csv` и т. Д. Сила pandas в основном заключается в возможности манипулировать большими объемами структурированных данных, поэтому мы хотим иметь возможность собрать всю необходимую информацию в одну таблицу, чтобы мы могли анализировать совокупные данные.

Мы можем совместить использование `glob`, библиотеки Python для работы с файлами, с пандасами, чтобы лучше организовать эти данные. `glob` может открывать несколько файлов, используя сопоставление регулярных выражений для получения имен файлов:

```
import glob

files = glob.glob("file*.csv")
```

```
df_list = []
for filename in files:
    data = pd.read_csv(filename)
    df_list.append(data)

df = pd.concat(df_list)

print(files)
```

Этот код проходит через любой файл, который начинается с 'file' и имеет расширение .csv. Он открывает каждый файл, считывает данные в DataFrame, а затем объединяет все эти DataFrames вместе.

Задание

1. У нас есть 5 разных файлов по 4 студентов в каждом. Эти файлы соответствуют структуре именования:

stud0.csv

stud1.csv

До stud 4.csv

Мы собираемся импортировать каждый файл с помощью pandas и объединить все записи в один DataFrame.

Сначала создайте переменную с именем student_files и установите ее равной glob () всех файлов csv, которые мы хотим импортировать.

2. Создайте пустой список с именем df_list, в котором будут храниться все фреймы данных, которые мы создаем из файлов stud0.csv по stud5.csv.
3. Прокрутите имена файлов в student_files и создайте DataFrame из каждого файла. Добавьте этот DataFrame в df_list.
4. Объедините все фреймы данных в df_list в один фрейм данных, называемый студентами.
5. Выведите в консоль students и его размер.

Работа с дубликатами

Часто мы видим дублирующиеся строки данных во фреймах данных, с которыми мы работаем. Это может произойти из-за ошибок при сборе данных или при сохранении и загрузке данных.

Для проверки на наличие дубликатов мы можем использовать функцию `pandas.duplicated()`, которая вернет ряд, указывающий нам, какие строки являются повторяющимися строками.

Допустим, у нас есть фрейм данных `fruits`, который представляет эту таблицу:

<i>item</i>	<i>price</i>	<i>calories</i>
"banana"	"\$1"	105
"apple"	"\$0.75"	95
"apple"	"\$0.75"	95
"peach"	"\$3"	55
"peach"	"\$4"	55
"clementine"	"\$2.5"	35

Если мы вызовем `fruits.duplicated()`, мы получим следующую таблицу:

<i>id</i>	<i>value</i>
0	False
1	False
2	True
3	False
4	False
5	False

Мы видим, что строка 2, представляющая "яблоко" с ценой "0,75 доллара" и 95 калориями, является дублирующей строкой. Каждое значение в этой строке совпадает с значением в другой строке.

Мы можем использовать функцию `pandas.drop_duplicates()`, чтобы удалить все строки, которые являются дубликатами другой строки.

Если мы вызовем `fruits.drop_duplicates()`, мы получим таблицу:

<i>item</i>	<i>price</i>	<i>calories</i>
"banana"	"\$1"	105
"apple"	"\$0.75"	95
"peach"	"\$3"	55
"peach"	"\$4"	55
"clementine"	"\$2.5"	35

Строка "яблоко" была удалена, потому что она была точно такой же, как и другая строка. Но две строки "персик" остаются, потому что в столбце цена есть разница.

Если бы мы хотели удалить каждую строку с повторяющимся значением в столбце "*item*", мы могли бы указать подмножество:

```
fruits = fruits.drop_duplicates(subset=['item'])
```

По умолчанию при этом сохраняется первое вхождение дубликата:

<i>item</i>	<i>price</i>	<i>calories</i>
"banana"	"\$1"	105
"apple"	"\$0.75"	95
"peach"	"\$3"	55
"clementine"	"\$2.5"	35

Убедитесь, что столбцы, из которых вы удаляете дубликаты, являются именно теми, в которых не могут быть дубликаты. Вы бы не хотели удалять дубликаты со столбцом цены в качестве подмножества, например, потому что это нормально, если несколько товаров стоят одну и ту же сумму!

Задание

1. Похоже, что в процессе сбора данных некоторые строки могли быть записаны дважды. Используйте функцию `.duplicated()` в фрейме данных `students`, чтобы создать объект серии, называемый дубликатами.
2. Распечатайте значение `.value_counts()` серии дубликатов, чтобы узнать, сколько строк являются точными дубликатами.
3. Обновите значение " `students` ", чтобы оно стало таблицей " `students` " с удалением дубликатов.
4. Снова используйте функцию `.duplicated()`, чтобы создать объект серии, называемый дубликатами, после удаления дубликатов. Распечатайте подсчеты значений еще раз.

Разделение по индексу

Пытаясь получить чистые данные, мы хотим убедиться, что каждый столбец представляет один тип измерения. Часто в одном столбце записывается несколько измерений, и мы хотим разделить их, чтобы можно было проводить индивидуальный анализ по каждой переменной.

Допустим, у нас есть столбец "день рождения" с данными, отформатированными в формате MMDDYYYY. Другими словами, "11011993" означает день рождения 1 ноября 1993 года. Мы хотим разделить эти данные на день, месяц и год, чтобы мы могли использовать эти столбцы как отдельные функции.

В этом случае мы знаем точную структуру этих строк. Первые два символа всегда будут соответствовать месяцу, вторые два - дню, а остальная часть строки всегда будет соответствовать году. Мы можем легко разбить данные на три отдельных столбца, разделив строки с помощью `.str`:

```
# Create the 'month' column
df['month'] = df.birthday.str[0:2]

# Create the 'day' column
df['day'] = df.birthday.str[2:4]

# Create the 'year' column
df['year'] = df.birthday.str[4:]
```

Первая команда берет первые два символа каждого значения в столбце "день рождения" и помещает их в столбец "месяц". Вторая команда берет вторые два символа каждого значения в столбце "день рождения" и помещает их в столбец "день". Третья команда берет оставшуюся часть каждого значения в столбце "день рождения" и помещает его в столбец "год".

Это привело бы к преобразованию таблицы, такой как:

<i>id</i>	<i>birthday</i>
1011	"12241989"
1112	"10311966"
1113	"01052011"

в таблицу вроде:

<i>id</i>	<i>birthday</i>	<i>month</i>	<i>day</i>	<i>year</i>
1011	"12241989"	"12"	"24"	"1989"
1112	"10311966"	"10"	"31"	"1966"
1113	"01052011"	"01"	"05"	"2011"

В одном из будущих упражнений мы попрактикуемся в преобразовании строковых столбцов в числовые (например, преобразование «10» в 10).

Задание

1. Выведите колонки students
2. Похоже, что столбец «gender_age» содержит и пол, и возраст!

Выведите .head () столбца, чтобы увидеть, какие данные он содержит.

3. Похоже, что первый символ значений в поле gender_age содержит пол, а остальная часть строки содержит возраст. Давайте выделим гендерные данные в новый столбец под названием «gender».
4. Теперь выделите данные о возрасте в новый столбец под названием age.
5. Молодец! Давайте выведем .head () студентов, чтобы увидеть, как DataFrame будет выглядеть после создания нами новых столбцов.
6. Теперь нам больше не нужен столбец «gender _ age».

Давайте установим DataFrame учащихся как DataFrame учащихся со всеми столбцами, кроме gender _ возраста.

Разделение по символам

Допустим, у нас есть столбец под названием «type» с записями данных в формате «admin_US» или «user_Kenya». Как мы видели раньше, этот столбец на самом деле содержит два типа данных. Один кажется типом пользователя (со значениями, такими как

«admin» или «user»), а другой - страной, в которой находится этот пользователь (со значениями, такими как «US» или «Kenya»).

Мы больше не можем просто разделить по первым 4 символам, потому что admin и user имеют разную длину. Вместо этого мы знаем, что хотим разделить по «_». Используя это, мы можем разделить этот столбец на два отдельных, более чистых столбца:

```
# Create the 'str_split' column
df['str_split'] = df.type.str.split('_')

# Create the 'usertype' column
df['usertype'] = df.str_split.str.get(0)

# Create the 'country' column
df['country'] = df.str_split.str.get(1)
```

Это преобразовало бы таблицу как:

<i>id</i>	<i>type</i>
1011	"user_Kenya"
1112	"admin_US"
1113	"moderator_UK"

в таблицу вроде:

<i>id</i>	<i>type</i>	<i>country</i>	<i>usertype</i>
1011	"user_Kenya"	"Kenya"	"user"
1112	"admin_US"	"US"	"admin"
1113	"moderator_UK"	"UK"	"moderator"

Задание

1. Имена студентов хранятся в столбце `full_name`.

Мы хотим разделить эти данные на два новых столбца, `first_name` и `last_name`.

Во-первых, давайте создадим объект `Series` с именем `name_split`, который разделяет полное имя на символ " ".

2. Теперь давайте создадим столбец с именем `first_name`, который принимает первый элемент в `name_split`.
3. Наконец, давайте создадим столбец с именем `last_name`, который принимает второй элемент в `name_split`.
4. Вывести `head()` таблицы `students`

Внимание на типы

Каждый столбец `DataFrame` может содержать элементы одного и того же типа данных или `dtype`. Панды используют следующие типы `dtypes`: `float`, `int`, `bool`, `datetime`, `timedelta`, `category` и `object`. Часто мы хотим преобразовать типы, чтобы лучше анализировать. Если числовая категория, такая как «`num_users`», хранится как Серия объектов, а не как целые числа, например, это затрудняет выполнение чего-то вроде построения линейного графика пользователей с течением времени.

Чтобы увидеть типы каждого столбца `DataFrame`, мы можем использовать:

```
print(df.dtypes)
```

Для такого `DataFrame`:

<i>item</i>	<i>price</i>	<i>calories</i>
"banana"	"\$1"	105
"apple"	"\$0.75"	95
"peach"	"\$3"	55
"clementine"	"\$2.5"	35

атрибут `.dtypes` будет:

```
item      object
price     object
calories  int64
dtype: object
```

Мы видим, что `dtype` самого атрибута `dtypes` является объектом! Это объект `Series`, с которым вы уже работали. Объекты серии составляют все фреймы данных.

Мы видим, что столбец цен состоит из объектов, что, вероятно, затруднит анализ цены. В следующих нескольких упражнениях мы рассмотрим, как преобразовать столбцы в числовые значения.

Задание

1. Давайте проверим типы в таблице студентов.

Выведите атрибут `.dtypes`.

2. Если бы мы хотели построить диаграмму рассеяния возраста и среднего балла на экзамене, сможем ли мы сделать это с этим типом данных?

Попробуйте распечатать среднее значение в столбце оценок учащихся.

Разбор строк

Иногда нам нужно изменить строки в наших `DataFrames`, чтобы преобразовать их в более значимые показатели. Например, в нашей таблице фруктов ранее:

<i>item</i>	<i>price</i>	<i>calories</i>
"banana"	"\$1"	105
"apple"	"\$0.75"	95
"peach"	"\$3"	55
"peach"	"\$4"	55
"clementine"	"\$2.5"	35

Мы видим, что столбец «цена» на самом деле состоит из строк, представляющих суммы в долларах. Этот столбец можно было бы гораздо лучше представить в виде чисел с плавающей запятой, чтобы мы могли взять среднее значение, вычислить другую совокупную статистику или сравнить разные фрукты друг с другом с точки зрения цены.

Во-первых, мы можем использовать то, что мы знаем о регулярных выражениях, чтобы избавиться от всех знаков доллара:

```
fruit.price = fruit['price'].replace('[\$',]', '', regex=True)
```

Затем мы можем использовать функцию pandas .to_numeric () для преобразования строк, содержащих числовые значения, в целые числа или числа с плавающей запятой:

```
fruit.price = pd.to_numeric(fruit.price)
```

Теперь у нас есть DataFrame, который выглядит так:

<i>item</i>	<i>price</i>	<i>calories</i>
"banana"	1	105
"apple"	0.75	95
"peach"	3	55
"peach"	4	55
"clementine"	2.5	35

Задание

1. В последнем упражнении мы увидели, что найти среднее значение в столбце оценки сложно, когда данные хранятся как объекты, а не числа.

Используйте регулярное выражение, чтобы убрать знаки% в столбце оценки.

2. Преобразуйте столбец оценки в числовой тип с помощью функции pd.to_numeric ().

Больше синтаксического анализа строк

Иногда мы хотим провести анализ чисел, скрытых в строковых значениях. Мы можем использовать регулярное выражение для извлечения этих числовых данных из строк, в которых они заключены. Предположим, у нас есть этот DataFrame df, представляющий тренировку:

<i>date</i>	<i>exerciseDescription</i>
10/18/2018	"lunges - 30 reps"
10/18/2018	"squats - 20 reps"
10/18/2018	"deadlifts - 25 reps"
10/18/2018	"jumping jacks - 30 reps"
10/19/2018	"lunges - 40 reps"
10/19/2018	"chest flyes - 15 reps"
...	...

Было бы полезно разделить такие данные, как «30 выпадов», на 2 столбца с количеством повторений «30» и типом упражнения «выпады». Затем мы могли бы сравнить, например, увеличение количества выполняемых выпадов с течением времени.

Чтобы извлечь числа из строки, мы можем использовать функцию pandas .str.split ():

```
split_df = df['exerciseDescription'].str.split('(\d+)', expand=True)
```

что приведет к этому DataFrame split_df:

<i>*</i>	<i>*</i>	<i>0</i>	<i>1</i>	<i>2</i>
0	"lunges - "		"30"	"reps"
1	"squats - "		"20"	"reps"
2	"deadlifts - "		"25"	"reps"

* *	0	1	2
3	"jumping jacks - "	"30"	"reps"
4	"lunges - "	"40"	"reps"
5	"chest flyes - "	"15"	"reps"
...	...	...	...

Затем мы можем назначить столбцы из этого DataFrame исходному df:

```
df.reps = pd.to_numeric(split_df[1])
df.exercise = split_df[2].replace('[\ -]', '', regex=True)
```

Теперь наш df выглядит так:

date	exerciseDescription	reps	exercise
10/18/2018	"lunges - 30 reps"	30	"lunges"
10/18/2018	"squats - 20 reps"	20	"squats"
10/18/2018	"deadlifts - 25 reps"	25	"deadlifts"
10/18/2018	"jumping jacks - 30 reps"	30	"jumping jacks"
10/19/2018	"lunges - 40 reps"	40	"lunges"
10/19/2018	"chest flyes - 15 reps"	15	"chest flyes"
...	...	...	...

Задание

1. Выведите первые пять строк столбца оценок.
2. Каждое значение в классе выглядит как "9th grade", "10th grade", "11th grade", "12th grade".

Мы хотим сократить это до числовой оценки. Возможно, мы хотим провести линейную регрессию по этим данным, для чего потребуются числовые входные данные.

Используйте регулярное выражение, чтобы извлечь число из каждой строки в оценке и сохранить эти значения обратно в столбец оценки.

3. Выведите типы таблицы students.
4. Преобразуйте столбец оценок в числовые значения, а не в объекты.
5. Вычислите среднее значение оценки, сохраните его в переменной avg_grade и затем распечатайте!

Мы не смогли бы этого сделать с такими строками, как "9th grade", "10th grade"

Пропущенные значения

У нас часто есть данные с отсутствующими элементами в результате проблемы с процессом сбора данных или ошибок в способе хранения данных. Отсутствующие элементы обычно отображаются как значения NaN (или Not a Number):

<i>day</i>	<i>bill</i>	<i>tip</i>	<i>num_guests</i>
"Mon"	10.1	1	1
"Mon"	20.75	5.5	2
"Tue"	19.95	5.5	NaN
"Wed"	44.10	15	3
"Wed"	NaN	1	1

Значение `num_gests` для 3-й строки отсутствует, а значение счета для 5-й строки отсутствует. Некоторые вычисления, которые мы делаем, просто пропускают значения `NaN`, но некоторые вычисления или визуализации, которые мы пытаемся выполнить, не работают при обнаружении `NaN`.

В большинстве случаев мы используем один из двух методов для работы с пропущенными значениями.

Метод 1: удалить все строки с отсутствующим значением

Для этого мы можем использовать `.dropna()`:

```
bill_df = bill_df.dropna()
```

Эта команда приведет к созданию `DataFrame` без неполных строк:

<i>day</i>	<i>bill</i>	<i>tip</i>	<i>num_guests</i>
"Mon"	10.1	1	1
"Mon"	20.75	5.5	2
"Wed"	44.10	15	3

Если бы мы хотели удалить каждую строку со значением `NaN` только в столбце `num_gests`, мы могли бы указать подмножество:

```
bill_df = bill_df.dropna(subset=['num_guests'])
```

Метод 2: заполните отсутствующие значения средним значением столбца или другим агрегированным значением.

Для этого мы можем использовать `.fillna()`:

```
bill_df = bill_df.fillna(value={"bill":bill_df.bill.mean(),
"num_guests":bill_df.num_guests.mean()})
```

Эта команда приведет к созданию `DataFrame` с соответствующим средним значением столбца вместо исходных `NaN`:

<i>day</i>	<i>bill</i>	<i>tip</i>	<i>num_guests</i>
"Mon"	10.1	1	1
"Mon"	20.75	5.5	2
"Tue"	19.95	5.5	1.75
"Wed"	44.10	15	3
"Wed"	23.725	1	1

Задание

1. Получите среднее значение в столбце оценки. Сохраните его в `score_mean` и выведите.
2. Предположим, что все, кто не набрал баллов за экзамен, его пропустили. Мы хотим заменить все `pans` значением 0. Давайте сделаем это со столбцом `Score`.

Заполните все пропуски в `['score']` 0.

3. Снова получите среднее значение из столбца оценок. Сохраните его в `score_mean_2` и выведите.

Заключение

Здорово! Мы рассмотрели ряд различных методов, которые мы можем использовать для преобразования данных в нужный нам формат для анализа.

В частности, мы рассмотрели:

- диагностика «аккуратности» данных
- изменение формы данных
- объединение нескольких файлов
- изменение типов ценностей
- удаление или заполнение отсутствующих значений - как мы поступаем с неполными или отсутствующими данными
- манипулирование строками для лучшего представления данных

Вы можете использовать эти методы для преобразования ваших наборов данных в чистые и удобные для работы!